

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Пермский национальный исследовательский политехнический
университет»

Электротехнический факультет

Кафедра: Информационные технологии и автоматизированные системы
(ИТАС)

Направление подготовки: 09.03.04 – «Программная инженерия»

ОТЧЕТ

Лабораторная работа №8.

Тема: «Программа, управляемая событиями»

По дисциплине «Основы алгоритмизации и программирования»

Вариант 13

Выполнила: студентка группы РИС-25-26

Лаптева Елена Владимировна

Принял: доц. Полякова О.А.

Пермь, 2026

Постановка задачи:

1. Определить иерархию пользовательских классов (см. лабораторную работу №5). Во главе иерархии должен стоять абстрактный класс с чисто виртуальными методами для ввода и вывода информации об атрибутах объектов.
2. Реализовать:
 - a. Конструкторы
 - b. Деструктор
 - c. Операцию присваивания
 - d. Селекторы и модификаторы
3. Определить класс-группу на основе структуры, указанной в варианте.
4. Для группы реализовать:
 - a. Конструкторы
 - b. Деструктор
 - c. Методы для добавления и удаления элементов в группу
 - d. Метод для просмотра группы
 - e. Перегрузить операцию для получения информации о размере группы
5. Определить класс Диалог – наследника группы, в котором реализовать методы для обработки событий.
6. Добавить методы для обработки событий группой и объектами пользовательских классов.
7. Написать тестирующую программу.
8. Нарисовать диаграмму классов и диаграмму объектов.

Базовый класс: ЧЕЛОВЕК (Person)

- Имя – string
- Возраст – int

Производный класс: АБИТУРИЕНТ (Abiturient)

- Количество баллов – int
- Специальность – string
- Группа – Tree

Команды для работы с группой:

- m - Создать группу (формат: m количество_элементов)
- + - Добавить элемент в группу
- - - Удалить элемент из группы
- s - Вывести информацию об элементах группы
- z - Вывести информацию о среднем возрасте
- q - Конец работы

Анализ задачи:

1. Создаем иерархию классов:
 - a. Абстрактный класс Person с чисто виртуальными методами showInfo(), readInfo()
 - b. Производный класс Abiturient с полями "баллы", "специальность"
 - c. Класс-контейнер (группа) для хранения указателей на Person (например, Tree или List)
 - d. Класс Dialog (наследник группы) с методами обработки команд пользователя (меню: m, +, -, s, z, q)
2. В main() создаем объект Dialog и демонстрируем работу всех команд через консольное меню.

Код:

Object.h

```
#pragma once
#include <string>
#include <iostream>
using namespace std;

struct TEvent;

class Object {
public:
    Object();
    virtual ~Object();

    // Чисто виртуальные методы
    virtual void Show() = 0;
    virtual void Input() = 0;
    virtual void HandleEvent(const TEvent& e) = 0;
};
```

Object.cpp

```
#include "Object.h"

Object::Object() {}

Object::~~Object() {}
```

Person.h

```
#pragma once

#include "Object.h"

class Person : public Object {
protected:
    string name;
    int age;

public:
    Person();
    Person(string n, int a);
    virtual ~Person();

    // Реализация виртуальных методов
    void Show() override;
    void Input() override;
    void HandleEvent(const TEvent& e) override;

    // Селекторы
    string GetName() const { return name; }
    int GetAge() const { return age; }

    // Модификаторы
    void SetName(string n) { name = n; }
    void SetAge(int a) { age = a; }

    // Оператор присваивания
    Person& operator=(const Person& p);
};
```

Person.cpp

```
#include "Person.h"
#include "Event.h"

Person::Person() : name(""), age(0) {}

Person::Person(string n, int a) : name(n), age(a) {}
```

```

Person::~Person() {}

void Person::Show() {
    cout << "Name: " << name << ", Age: " << age;
}

void Person::Input() {
    cout << "Enter name: ";
    cin >> name;
    cout << "Enter age: ";
    cin >> age;
}

void Person::HandleEvent(const TEvent& e) {
    // Базовый класс не обрабатывает события
}

Person& Person::operator=(const Person& p) {
    if (this != &p) {
        name = p.name;
        age = p.age;
    }
    return *this;
}

```

Abiturient.h

```

#pragma once

#include "Person.h"

class Abiturient : public Person {
private:
    int score;           // Количество баллов
    string specialty;    // Специальность

public:
    Abiturient();
    Abiturient(string n, int a, int s, string spec);
}

```

```

~Abiturient();

// Переопределение виртуальных методов
void Show() override;
void Input() override;
void HandleEvent(const TEvent& e) override;

// Селекторы
int GetScore() const { return score; }
string GetSpecialty() const { return specialty; }

// Модификаторы
void SetScore(int s) { score = s; }
void SetSpecialty(string spec) { specialty = spec; }

// Оператор присваивания
Abiturient& operator=(const Abiturient& a);
};

```

Abiturient.cpp

```

#include "Abiturient.h"
#include "Event.h"

Abiturient::Abiturient() : Person(), score(0), specialty("") {}

Abiturient::Abiturient(string n, int a, int s, string spec)
    : Person(n, a), score(s), specialty(spec) {}

Abiturient::~Abiturient() {}

void Abiturient::Show() {
    cout << "Abiturient: ";
    Person::Show();
    cout << ", Score: " << score << ", Specialty: " << specialty;
}

void Abiturient::Input() {

```

```

    Person::Input();
    cout << "Enter score: ";
    cin >> score;
    cout << "Enter specialty: ";
    cin >> specialty;
}

void Abiturient::HandleEvent(const TEvent& e) {
    // Специфичная обработка
}

Abiturient& Abiturient::operator=(const Abiturient& a) {
    if (this != &a) {
        Person::operator=(a);
        score = a.score;
        specialty = a.specialty;
    }
    return *this;
}

```

Event.h

```

#pragma once

// Константы событий
const int evNothing = 0;    // Пустое событие
const int evMessage = 100; // Событие-сообщение

// Коды команд
const int cmAdd = 1;        // Добавить объект
const int cmDel = 2;        // Удалить объект
const int cmShow = 3;       // Показать группу
const int cmMake = 4;       // Создать группу
const int cmQuit = 5;       // Выход
const int cmAvgAge = 6;     // Средний возраст

// Структура события
struct TEvent {
    int what; // Тип события
}

```

```

union {
    int command; // Код команды
    struct {
        int message;
        int a; // Параметр команды
    };
};
};

```

Tree.h

```

#pragma once
#include "Object.h"
#include "Event.h"

// Узел дерева
struct TreeNode {
    Object* data;
    TreeNode* left;
    TreeNode* right;

    TreeNode(Object* obj) : data(obj), left(nullptr), right(nullptr) {}
};

class Tree {
protected:
    TreeNode* root;
    int size;
    int count; // Текущее количество элементов

    // Вспомогательные методы для работы с деревом
    void ClearTree(TreeNode* node);
    void ShowTree(TreeNode* node);
    void DeleteTree(TreeNode* node);
    int CalculateTotalAge(TreeNode* node);
    int CountNodes(TreeNode* node);

public:

```



```

Tree();
Tree(int n);
~Tree();

// Методы управления
void Add(Object* obj);
void Delete();
void Show();

// Перегрузка операции
int operator()(); // Размер группы

// Обработка событий
virtual void HandleEvent(const TEvent& e);

// Вычисление среднего возраста
double GetAverageAge();
};

```

Tree.cpp

```

#include "Tree.h"
#include "Person.h"
#include "Abiturient.h"
#include <iostream>
using namespace std;

Tree::Tree() : root(nullptr), size(0), count(0) {}

Tree::Tree(int n) : root(nullptr), size(n), count(0) {}

Tree::~Tree() {
    ClearTree(root);
}

void Tree::ClearTree(TreeNode* node) {
    if (node) {
        ClearTree(node->left);
        ClearTree(node->right);
    }
}

```

```

        delete node->data;
        delete node;
    }
}

void Tree::Add(Object* obj) {
    if (count >= size) {
        cout << "Tree is full!\n";
        return;
    }

    TreeNode* newNode = new TreeNode(obj);

    if (!root) {
        root = newNode;
    }
    else {
        TreeNode* current = root;
        TreeNode* parent = nullptr;

        Person* newPerson = dynamic_cast<Person*>(obj);

        while (current) {
            parent = current;
            Person* currentPerson = dynamic_cast<Person*>(current->data);

            if (currentPerson && newPerson) {
                if (newPerson->GetAge() < currentPerson->GetAge())
                    current = current->left;
                else
                    current = current->right;
            }
            else {
                break;
            }
        }

        if (parent) {

```

```

        Person* parentPerson = dynamic_cast<Person*>(parent->data);
        if (parentPerson && newPerson) {
            if (newPerson->GetAge() < parentPerson->GetAge())
                parent->left = newNode;
            else
                parent->right = newNode;
        }
    }
}

count++;
cout << "Element added to tree.\n";
}

```

```

void Tree::Delete() {
    if (!root) {
        cout << "Tree is empty!\n";
        return;
    }

    TreeNode* current = root;
    TreeNode* parent = nullptr;

    while (current->right) {
        parent = current;
        current = current->right;
    }

    if (parent)
        parent->right = nullptr;
    else
        root = nullptr;

    delete current->data;
    delete current;
    count--;

    cout << "Element deleted from tree.\n";
}

```

```
}
```

```
void Tree::ShowTree(TreeNode* node) {
```

```
    if (node) {  
        node->data->Show();  
        cout << endl;  
        ShowTree(node->left);  
        ShowTree(node->right);  
    }
```

```
}
```

```
void Tree::Show() {
```

```
    if (!root) {  
        cout << "Tree is empty.\n";  
        return;  
    }
```

```
    cout << "Tree elements (" << count << "):\n";  
    ShowTree(root);
```

```
}
```

```
int Tree::operator()() {
```

```
    return count;
```

```
}
```

```
int Tree::CalculateTotalAge(TreeNode* node) {
```

```
    if (!node) return 0;  
    Person* p = dynamic_cast<Person*>(node->data);  
    int age = p ? p->GetAge() : 0;  
    return age + CalculateTotalAge(node->left) + CalculateTotalAge(node->right);
```

```
}
```

```
int Tree::CountNodes(TreeNode* node) {
```

```
    if (!node) return 0;  
    return 1 + CountNodes(node->left) + CountNodes(node->right);
```

```
}
```

```
double Tree::GetAverageAge() {
```

```

        if (!root) return 0.0;
        int totalAge = CalculateTotalAge(root);
        int nodes = CountNodes(root);
        return nodes > 0 ? static_cast<double>(totalAge) / nodes : 0.0;
    }

```

```

void Tree::HandleEvent(const TEvent& e) {
    // Implementation if needed
}

```

Dialog.h

```

#pragma once
#include "Tree.h"
#include "Event.h"

class Dialog : public Tree {
private:
    int EndState;

public:
    Dialog();
    Dialog(int n);
    ~Dialog();

    // Основные методы обработки событий
    virtual void GetEvent(TEvent& event);
    virtual int Execute();
    virtual void HandleEvent(TEvent& event);
    virtual void ClearEvent(TEvent& event);

    // Проверка состояния
    int Valid();
    void EndExec();
};

```

Dialog.cpp

```

#include "Dialog.h"
#include "Person.h"
#include "Abiturient.h"

```

```

#include <iostream>
#include <string>
using namespace std;

Dialog::Dialog() : Tree(), EndState(0) {}

Dialog::Dialog(int n) : Tree(n), EndState(0) {}

Dialog::~Dialog() {}

void Dialog::GetEvent(TEvent& event) {
    string validOps = "+-smqaz";
    string input;
    char code;

    cout << "> ";
    cin >> input;

    if (input.empty()) {
        event.what = evNothing;
        return;
    }

    code = input[0];

    if (validOps.find(code) != string::npos) {
        event.what = evMessage;

        switch (code) {
            case 'm':
                event.command = cmMake;
                if (input.length() > 1) {
                    int sz = stoi(input.substr(1));
                    event.a = sz;
                }
                break;
            case '+':
                event.command = cmAdd;

```

```

        break;
    case '-':
        event.command = cmDel;
        break;
    case 's':
        event.command = cmShow;
        break;
    case 'z':
        event.command = cmAvgAge;
        break;
    case 'q':
        event.command = cmQuit;
        break;
    }
}
else {
    event.what = evNothing;
}
}

int Dialog::Execute() {
    TEvent event;

    do {
        EndState = 0;
        GetEvent(event);
        HandleEvent(event);

        if (event.what != evNothing) {
            cout << "Unhandled event!\n";
        }
    } while (!Valid());

    return EndState;
}

void Dialog::HandleEvent(TEvent& event) {
    if (event.what == evMessage) {

```

```

switch (event.command) {
case cmMake:
    size = event.a;
    cout << "Tree created with size " << size << "\n";
    ClearEvent(event);
    break;

case cmAdd: {
    cout << "1. Person\n2. Abiturient\nChoice: ";
    int choice;
    cin >> choice;

    Object* obj = nullptr;

    if (choice == 1) {
        Person* p = new Person();
        p->Input();
        obj = p;
    }
    else if (choice == 2) {
        Abiturient* a = new Abiturient();
        a->Input();
        obj = a;
    }

    if (obj) {
        Add(obj);
    }
    ClearEvent(event);
    break;
}

case cmDel:
    Delete();
    ClearEvent(event);
    break;

case cmShow:

```



```

        Show();
        ClearEvent(event);
        break;

    case cmAvgAge:
        cout << "Average age: " << GetAverageAge() << "\n";
        ClearEvent(event);
        break;

    case cmQuit:
        EndExec();
        ClearEvent(event);
        break;

    default:
        Tree::HandleEvent(event);
        break;
    }
}
}

```

```

void Dialog::ClearEvent(TEvent& event) {
    event.what = evNothing;
}

```

```

int Dialog::Valid() {
    return EndState != 0;
}

```

```

void Dialog::EndExec() {
    EndState = 1;
}

```

main.cpp

```

#include "Dialog.h"
#include <iostream>
using namespace std;

```

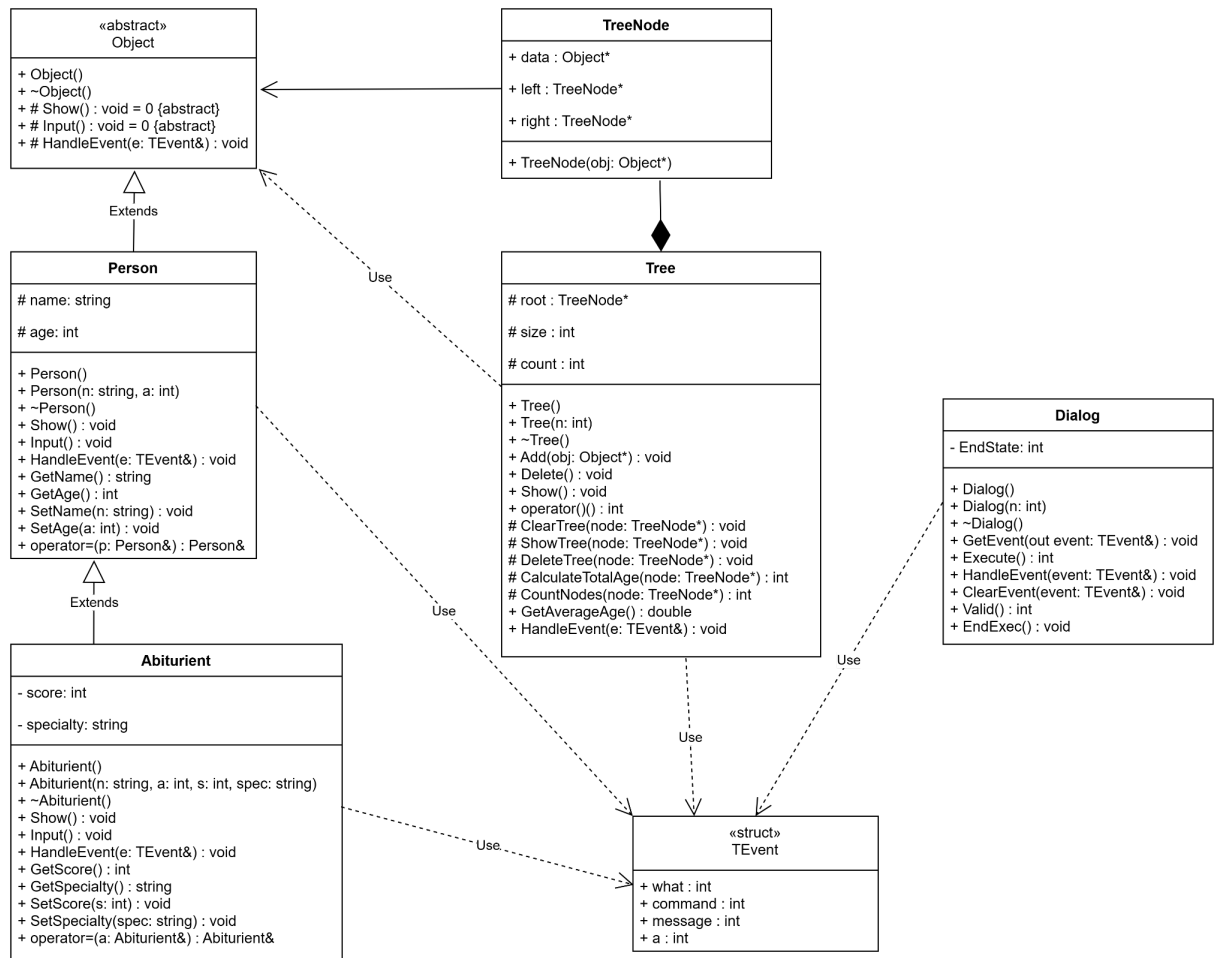
```
int main() {  
    setlocale(LC_ALL, "Russian");  
  
    cout << "ПРОГРАММА, УПРАВЛЯЕМАЯ СОБЫТИЯМИ" << endl;  
    cout << "Вариант 13: Абитуриенты (Дерево)" << endl;  
    cout << endl;  
    cout << "Доступные команды:" << endl;  
    cout << "  mN  - Создать группу (N - размер)" << endl;  
    cout << "  +   - Добавить элемент" << endl;  
    cout << "  -   - Удалить элемент" << endl;  
    cout << "  s   - Показать все элементы" << endl;  
    cout << "  z   - Средний возраст" << endl;  
    cout << "  q   - Выход" << endl;  
    cout << endl;  
  
    Dialog app(10);  
    app.Execute();  
  
    cout << "Программа завершена." << endl;  
    return 0;  
}
```

Результаты работы:

```
ПРОГРАММА, УПРАВЛЯЕМАЯ СОБЫТИЯМИ
Вариант 13: Абитуриенты (Дерево)
}
Доступные команды:
mN - Создать группу (N - размер)
+ - Добавить элемент
- - Удалить элемент
s - Показать все элементы
z - Средний возраст
q - Выход

> m5
Tree created with size 5
> +
1. Person
2. Abiturient
Choice: 1
Enter name: Rayan
Enter age: 13
Element added to tree.
> s
Tree elements (1):
Name: Rayan, Age: 13
> +
1. Person
2. Abiturient
Choice: 1
Enter name: Tea
Enter age: 12
Element added to tree.
> я
> я
> я
> z
Average age: 12.5
> -
Element deleted from tree.
> s
Tree is empty.
>
```

UML:



Вопросы:

1. Класс-группа – это объект, в который включены другие объекты (элементы группы). Примеры: окно с элементами (поля ввода, кнопки), агрегат из узлов, огород с растениями, факультет с кафедрами.

2. Класс-группа **List** хранит указатели на объекты базового класса **Object** в виде связного списка, управляет их жизненным циклом и предоставляет методы для работы с группой:

```
class List {
```

```
protected:
```

```
    struct Node { Object* data; Node* next; };
```

```
    Node* head; // указатель на начало списка
```

```
    int size; // максимальный размер
```

```
    int count; // текущее количество элементов
```

public:

```
List(int n);      // конструктор с параметром
List();           // конструктор без параметров
~List();          // деструктор
void Add(Object* obj); // добавление элемента
void Del();        // удаление элемента
void Show();       // просмотр группы
int operator()();  // возврат текущего размера
};
```

3. Для класса-группы Вектор: конструктор с параметром: `Vector(int n) { beg=new Object*[n]; cur=0; size=n; }`. Конструктор без параметров в тексте не приведён для `Vector`, но есть для `Dialog`: `Dialog(void):Vector() { EndState=0; }`. Конструктор копирования не описан, но по аналогии можно предположить, что он должен копировать указатели.

4. Деструктор для класса-группы `Vector`: `Vector::~~Vector(void) { if(beg!=0) delete [] beg; beg=0; }`

5. Метод просмотра элементов для `Vector`: `void Vector::Show() { if(cur==0) cout<<"Empty"<<endl; Object **p=beg; for(int i=0;i<cur;i++) { (*p)->Show(); p++; } }`

6. Группа даёт иерархию объектов (иерархию типа целое/часть), построенную на основе агрегации. Первый вид иерархии – иерархия классов на основе наследования.

7. Абстрактный класс нужен, чтобы обеспечить полиморфизм: методы `Show`, `Input`, `HandleEvent` вызываются через указатели на базовый класс, а реальные объекты могут быть разных производных типов (`Car`, `Lorry`). Чисто виртуальные функции заставляют производные классы реализовать их.

8. Событие – это пакет информации, которым обмениваются объекты. Создаётся средой в ответ на действия пользователя. События управляют работой объектов: в ответ на событие создаются, модифицируются или уничтожаются объекты.

9. Событие-сообщение должно иметь: код класса сообщения (отличает сообщения объектов одного класса от другого), адрес объекта-получателя (может отсутствовать), информационное поле.

10. Пример структуры события: `struct TEvent { int what; union { int command; struct { int message; int a; }; }; }`

11. Поле `what` определяет тип события и принимает два основных значения:

- `evNothing (0)` — пустое событие. Присваивается, когда ничего делать не нужно, или после того как объект уже обработал событие (чтобы оно не обрабатывалось повторно).
- `evMessage (100)` — событие-сообщение от объекта. Присваивается, когда поступила команда или данные, требующие обработки (например, ввод команды с клавиатуры).

В расширенных структурах (с мышью/клавиатурой) `what` также указывает источник события (мышь, клавиатура, системное сообщение).

12. Полю `command` присваиваются значения констант: `cmAdd (1)`, `cmDel (2)`, `cmGet (3)`, `cmShow (4)`, `cmMake (6)`, `cmQuit (101)` и т.д., в зависимости от введённой пользователем команды.

13. Поля `a` и `message` используются для передачи параметров команды. Например, выделение параметра из строки команды: `event.a = A` (число). В событии от объекта (`evMessage`) задаются два параметра: `command` (код команды) и `message` (передаваемая информация).

14. Необходимы методы: `GetEvent` (формирование события), `Execute` (главный цикл обработки), `HandleEvent` (обработчик), `ClearEvent` (очистка события). Также методы `Valid`, `EndExec`.

15. Главный цикл обработки событий: `int TMyApp::Execute() { do { GetEvent(event); HandleEvent(event); if(event.what!=evNothing) EventError(event); } while(!Valid()); return endState; }`

16. `ClearEvent` очищает событие, присваивая полю `event.what` значение `evNothing`. Это делается, чтобы событие не обрабатывалось далее.

17. `HandleEvent` обрабатывает каждое событие. Для объекта он сначала вызывает обработчик базового класса, затем проверяет тип события и команду, выполняет соответствующие действия и очищает событие. Для группы – сначала обрабатывает команды группы, затем, если событие не обработано, передаёт его элементам.

18. `GetEvent` получает событие от пользователя (или системы). В программе он считывает команду с клавиатуры, анализирует её, заполняет структуру `TEvent` (`what`, `command`, `a`).

19. EndState – это protected-поле класса Dialog, используется для завершения работы программы. Метод EndExec() устанавливает EndState=1, а метод Valid() возвращает true, если EndState!=0.

20. Valid() проверяет атрибут EndState. Возвращает true, если EndState не 0 (т.е. пришла команда выхода). Для группы Valid может вызывать Valid своих элементов, чтобы завершить работу только когда все элементы готовы.